COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	OCTOBER 6 2026	**DATE SUBMITTED:**	OCTOBER 6 2026
NAME:	JARED ISAIAH P. DELA CRUZ DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QEUJADO

ACTIVITY NO. 11.1 **Hands-on Activity 11.1 Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

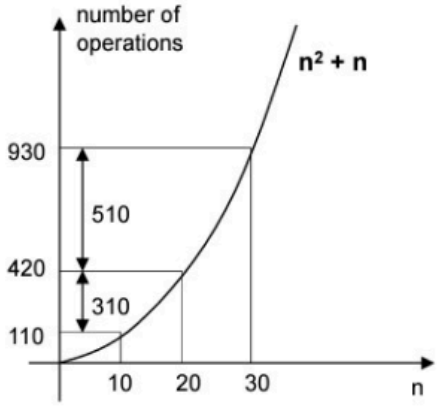
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

ANSWER: m x n or mn. In the worst case, the function is called to check every element of the second array (y) against the first one (x) for confirmation. For each iteration, search(x, y[i]) is called, which performs m comparisons in the worst case to check against all m elements of array x. Multiplying the n loop iterations by the m comparisons performed each search gives the answer we're looking for.

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

ANALYSIS: The graph displays a quadratic curve because it is n^2 + n. Moreover, the worst case for this algorithm is O(n^2). It is because it shows that the array size n increases, and the number of operations grows at a non linear rate.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the
The department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

First Name	Last Name	Flu Shot?
1	3	Yes
2	2	No
2	3	Yes

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

Time taken to search = 45 microseconds
Student (836 118) needs vaccination.
Time taken to search = 37 microseconds
Student (836 118) needs vaccination.
Time taken to search = 53 microseconds
Student (836 118) needs vaccination.

The high-level steps for this implementation are as follows:
1. Represent each student as an object of the Student class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL std::sort() function.
3. Use a binary search to see if the student is present on the list.
4. If the student isn't present in the list, your function should return true since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. Else, return false

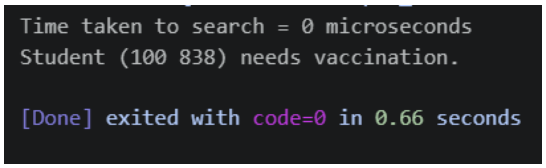
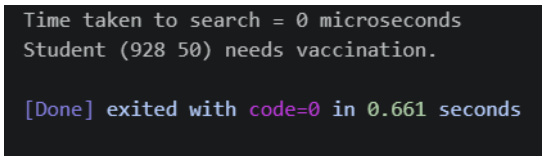
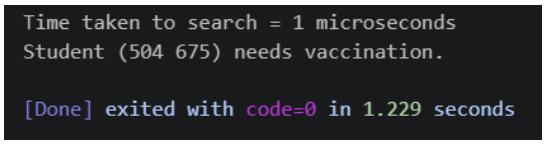
Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0.66 seconds		The input size N is the smallest compared to the other inputs, which results in a faster exit.
10000	0.661 seconds		The input size N is in the middle ground between the three given inputs, which gave out a medium speed of exit
100000	1.229 seconds		Because of the relatively large input size, the program exited much slower, around twice as much as the previous ones.

Table 11-1. Results

ANALYSIS: As we executed the given program, we ran through the code lines carefully to determine how our outputs came to be. Firstly, it checks for the student information using a random list using generate_random_Student function code. Afterwards, using the steady_clock function line, we begin to tally the total time it takes to execute the program with the given input size. After placing the function, we can now seek which students need vaccination. Finally, after returning if the given random student needs vaccination or not, the execution time is recorded, and it comes with the program output in the end. Turns out that the bigger the

given search_test value was, the more time it took to return or exit the output each time.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique.

CODE:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>

bool is_unique(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

void run_experiment(int size) {
    std::vector<int> A(size);
    for (int i = 0; i < size; i++) {
        A[i] = i;
    }

    //measure execution time
    auto start = std::chrono::high_resolution_clock::now();
    bool result = is_unique(A);
    auto end = std::chrono::high_resolution_clock::now();

```

```cpp
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();
    std::cout << "Input Size (N = " << size << "): " << duration << " microseconds" << std::endl;
}

int main() {
    std::cout << "Problem 1: Experimental Analysis" << std::endl;
    run_experiment(1000);
    run_experiment(5000);
    run_experiment(10000);
    run_experiment(20000);
    return 0;
}
```

OUTPUTS:

THEORETICAL ANALYSIS:

Mapas: In this algorithm, it uses a nested loop to compare every pair of elements in the array. Moreover, the worst case would be n(n-1) / 2 comparisons thus resulting in a time complexity of O(n^2). Lastly, the total memory consumes 4n + 8 bytes since an integer is a 4 byte.

Dela Cruz: The given program tests if all the elements in Array A differ in size n by comparing each unique pairings. Worst Case would occur when all elements in the array are unique, in which it is n(n-1) / 2 in comparisons, which will result in a $O(n^2)$ time complexity.

EXPERIMENTAL ANALYSIS:

Mapas: In this algorithm, it was tested on different sizes and these are; 1000, 5000, 10000, and 20000 which yield a runtimes of 2095, 45870, 189874, 716054 microseconds respectively. Furthermore, and input size N increases the runtime also increases and from N = 10000 to 20000 the execution time nearly fourfold which indicates a quadratic growth.

```
Input Size (N = 1000): 2095 microseconds
Input Size (N = 5000): 45870 microseconds
Input Size (N = 10000): 189874 microseconds
Input Size (N = 20000): 716054 microseconds

[Done] exited with code=0 in 1.468 seconds
```

Dela Cruz: Using test times that are bigger each increment (1000, 5000, 10000, 20000), we can easily showcase the fact that increasing the input size each time should exponentially increase the given execution time factor. With this in mind, the Experimental Analysis further proves that the theoretical worst-case time complexity is valid.

```
Input Size (N = 1000): 6735 microseconds
Input Size (N = 5000): 190838 microseconds
Input Size (N = 10000): 621899 microseconds
Input Size (N = 20000): 2342603 microseconds
```

Analysis and Comparison

- **Dela Cruz:** With the given results through the two analyses, we can conclude that the experimental analysis helped validate the theoretical analysis case of worst case time complexity of $O(n^2)$ because the execution time exponentially increases when the given input size is bigger.

- **Mapas:** After observing all the results and creating the two analyses, It can be analyzed and compared that the experimental results match the theoretical expectations of O(n^2) because of a quadratic growth. Furthermore, there is also a minor mismatch between the two because of hardware instances. With that we can say that the execution times exponentially increases when the input size also increases.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

CODE:

```cpp
//Experimental Analysis

#include <iostream>
#include <chrono>

long long rpower(int x, int n) {
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}


long long brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) { // odd
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else { // even
        long long y = brpower(x, n / 2);
        return y * y;
    }
}

void run_power_experiment(int x, int n) {
    //measure rpower execution time
    auto start_r = std::chrono::high_resolution_clock::now();
    long long res_r = rpower(x, n);
    auto end_r = std::chrono::high_resolution_clock::now();
```

```cpp
    auto duration_r = std::chrono::duration_cast<std::chrono::nanoseconds>(end_r - start_r).count();

    //measure brpower execution time
    auto start_br = std::chrono::high_resolution_clock::now();
    long long res_br = brpower(x, n);
    auto end_br = std::chrono::high_resolution_clock::now();
    auto duration_br = std::chrono::duration_cast<std::chrono::nanoseconds>(end_br - start_br).count();

    std::cout << "Exponent N = " << n << std::endl;
    std::cout << " rpower (Linear O(N)) : " << duration_r << " ns" << std::endl;
    std::cout << " brpower (Log O(logN)): " << duration_br << " ns" << std::endl;
    std::cout << " " << std::endl;
}

int main() {
    std::cout << "Problem 2: Experimental Analysis" << std::endl;

    run_power_experiment(2, 100);
    run_power_experiment(2, 1000);
    run_power_experiment(2, 10000);
    run_power_experiment(2, 100000);

    return 0;
}
```

OUTPUTS:

Theoretical Analysis

Mapas: This simple recursive algorithm rpower reduces n by 1 each step, making n recursive calls for a time and a stack space complexity of O(n). However, the brpower halves n at each step, taking T(n) = T((n/2)) + O(1) which will yield a time complexity of O(log n). Lastly, since an integer is 4bytes, rpower uses 8n + 8 bytes of stack memory whereas brpower uses 8(log_2 n) + 8 bytes.

Dela Cruz: This algorithm showcases the rpower and bpower. For the rpower it executes recursive calls while the function decreases n by 1 per step. With this, it gives a O(n) time complexity. For the bpower, it halves n per step, making $[\log_2 n] + 1$ recursive calls, which then results in a time complexity of O(log n).

Experimental Analysis

Mapas: In this analysis both functions were tested with N = 100, 1000, and 10000 wherein rpower scaled linearly from 700 ns to 23700 ns and then to 260200 ns. However, brpower stayed fast from 100 ns, to 100 ns and then to 200 ns.

```
Problem 2: Experimental Analysis
Exponent N = 100
 rpower (Linear O(N)) : 700 ns
 brpower (Log O(logN)): 100 ns

Exponent N = 1000
 rpower (Linear O(N)) : 23700 ns
 brpower (Log O(logN)): 100 ns

Exponent N = 10000
 rpower (Linear O(N)) : 260200 ns
 brpower (Log O(logN)): 200 ns


[Done] exited with code=3221225725 in 0.409 seconds
```

-

Dela Cruz: I ran the similar program on the device available to me. For the rpower, it jumped from 800ns, 47900ns, and 357900ns as the Exponent N input increased. As for my brpower, the Exponent N 100 and 10000 returned with the same 200ns. It got me confused as how the Exponent N = 1000 returned 16200ns. Overall, however, it would seem that my device processed the algorithm significantly slower than my partner.

```
Problem 2: Experimental Analysis
Exponent N = 100
 rpower (Linear O(N)) : 800 ns
 brpower (Log O(logN)): 200 ns

Exponent N = 1000
 rpower (Linear O(N)) : 47900 ns
 brpower (Log O(logN)): 16200 ns

Exponent N = 10000
 rpower (Linear O(N)) : 357900 ns
 brpower (Log O(logN)): 200 ns
```

Analysis and Comparison

- **Dela Cruz:** With the outputs given in mind as proof of our analyses, we can conclude that the

experimental analysis validates our theoretical one. The difference between the processing method of rpower is slower since it goes one by one. Brpower seemed superior because it halves the given N each process, which improves overall efficiency and speed when it comes to large exponents or input.

- **Mapas:** The experimental analysis confirms our theoretical analysis. It is because rpower processes the exponent step by step, its execution time grows steadily with larger exponents. While brpower eliminates most of that work by repeatedly cutting N in half. Hence, brpower proves to be more fast and more efficient for large powers.

CONCLUSION

Dela Cruz

- This activity demonstrates how theoretical complexity directly accurately predicts real-world algorithm performance. For both problems, the experimental timing results closely matched the theoretical expectations: the $O(n^2)$ unique element algorithm showed quadratic runtime growth, while the power algorithms clearly proved that the logarithmic O(log n) approach is far faster and more efficient than the linear O(n) approach as the input size increases. Overall, the experiment confirms that choosing an algorithm with a better theoretical complexity is essential for writing efficient, scalable code.

Mapas

- In this activity, I learned about basic algorithm analysis. I also measured the runtime of algorithms using theoretical analysis. Moreover, I also analyze the results of runtime performance by comparing experimental and theoretical analysis. Furthermore, during our procedure, my pair and I answered every needed question and analysis while on the supplementary activity, we had our own individual analysis. Lastly, I think I still have a lot to learn from this topic, and I will keep learning in order to understand it more.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used gemini as a tool for understanding and to guide us in code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.